Введение

LLM (Large Language Model, большая языковая модель) – это продвинутый алгоритм искусственного интеллекта на базе нейронной сети, обученный на огромных массивах текстовых данных для понимания и генерации человеческого языка. Большие языковые модели считаются одним из наиболее значительных технологических прорывов последнего десятилетия. Однако у каждой мощной технологии есть своя теневая сторона, и в случае с LLM она проявляется в виде принципиально новых векторов киберугроз, с которыми традиционные средства защиты информации справляются далеко не всегда. Парадокс ситуации заключается в том, что модель, обученная быть максимально полезной, оказывается уязвимой для манипуляций: инструкция, замаскированная под безобидный вопрос, может заставить систему раскрыть конфиденциальные данные, обойти встроенные этические ограничения или открыть путь к реальным деструктивным действиям в цифровой инфраструктуре. Осознание этих рисков привело к формированию целой области исследований, находящейся на стыке машинного обучения, компьютерной безопасности и инженерии программного обеспечения. В настоящей статье предпринята попытка систематического рассмотрения проблемы безопасности LLM в трёх взаимосвязанных аспектах: анализ архитектурных и процедурных ограничений, классификация основные типы уязвимостей и рассмотрение актуальных методов защиты и повышения безопасности.

Ограничения в LLM

В современной практике разработки больших языковых моделей обеспечение безопасного поведения не является монолитным актом, совершаемым в одной точке жизненного цикла.

Напротив, ограничения встраиваются инженерно распределённым образом, формируя многослойную архитектуру. Каждый из этих слоёв решает свою специфическую задачу, однако каждый из них обладает и собственными структурными уязвимостями.

Первый слой формируют ограничения до развёртывания, предшествующие непосредственно обучению. На данном этапе закладываются ограничения, связанные с информационной основой модели, а именно о формировании тренировочного корпуса данных. В современных LLM реализуются многоступенчатые процедуры фильтрации, направленные на устранение вредоносного контента, который модель способна закрепить в своих весах. Параллельно с этим задаются архитектурные ограничения (размер контекстного окна, лимиты на количество вычислительных шагов при рассуждении), которые носят физический характер, то есть модель не способна учитывать информацию за пределами установленного горизонта внимания или тратить на «размышление» больше имеющихся вычислений. Однако здесь обнаруживается интересная особенность: модель, лишённая в своих весах определённых знаний, всё равно хорошо понимает логику инструкций, и если злоумышленник предоставит ей вредоносную инструкцию, то модель, не обладая этими знаниями, справится с их интерпретацией и выполнением.

Второй слой формируется в процессе обучения. Следующий и наиболее активный этап формирования ограничений приходится на стадию выравнивания модели. Здесь происходит целенаправленное изменение функции, которую модель аппроксимирует. Посредством методов обучения с подкреплением на основе обратной связи от человека или через формализованные политики безопасности, модель обучают не просто предсказывать следующее слово, а

предпочитать одни траектории генерации другим. В этот момент ограничения перестают быть внешними и становятся неотъемлемой частью параметрической памяти системы. Модель учится отказывать, переформулировать опасные запросы или уходить в нейтральные объяснения. Тем не менее, многочисленные эмпирические исследования демонстрируют хрупкость этой конструкции. Данный слой защиты оказался уязвим для атак типа jailbreak, которые заставляют модель игнорировать её поведение. Более того, в условиях открытого распространения весов этот слой и вовсе теряет смысл, поскольку злоумышленник может удалить или модифицировать соответствующие участки нейронной сети.

Третий слой – ограничения после развёртывания. Когда модель развёрнута и взаимодействует с пользователем, в игру вступает внешняя инфраструктура модерации. Речь идёт не о внутренних качествах модели, а о внешнем программном контуре, в который она помещена. Типичная архитектура включает премодерацию входящего запроса специальной моделью-классификатором, внедрение скрытых системных инструкций, задающих контекст поведения, и постмодерацию сгенерированного ответа с возможностью его блокировки или подмены. С точки зрения эксплуатации, это наиболее гибкий инструмент, позволяющий оперативно реагировать на новые угрозы без переобучения основной модели. Однако именно этот слой оказывается наиболее уязвимым. Логика постобработки текста полностью разрушается при интеграции модели с внешними инструментами. Если агент совершает вызов функции, вредоносное действие происходит не в момент генерации вербального отчёта, а в момент самого вызова API. В этой точке разрыва между действием и отчётом о нём современные системы надзора оказываются слепы.

Понимание природы ограничений является ключевым для дальнейшего развития области безопасного искусственного интеллекта, особенно в преддверии перехода к парадигмам непрерывного обучения, где все три описанных слоя рискуют слиться в единый, трудно контролируемый процесс.

Типы уязвимостей LLM

Но прежде чем говорить об этой будущей сложности, стоит оценить, какие опасности окружают нас прямо сейчас. Сегодняшние уязвимости LLM, при всём их многообразии, поддаются систематизации. Опираясь на актуальные исследования, зафиксированные в стандарте OWASP Top 10 for LLM Applications, можно выделить несколько фундаментальных категорий рисков, которые формируют слабые точки современных языковых моделей. Каждая из них эксплуатирует определённый аспект архитектуры взаимодействия человека и машины, и каждая требует собственной стратегии.

Главное место в этой классификации отводится явлению промпт-инъекции (Prompt Injection). Модель, согласно своей архитектуре, склонна рассматривать весь входящий поток токенов как единый контекст, не обладая врождённым иммунитетом к различению системной директивы и пользовательского шума. Вредоносная конструкция маскируется под обычный контент, заставляя модель игнорировать исходные правила поведения. Здесь важно различать прямые инъекции, когда атакующий напрямую пишет «забудь предыдущие указания», и непрямые, при которых вредоносная инструкция скрыта на веб-странице или в документе, который модель обрабатывает в фоновом режиме. В результате модель может непреднамеренно воспроизвести фрагменты конфиденциальных или запрещённых данных.

Отдельного рассмотрения заслуживает практика джейлбрейка (Jailbreak), эксплуатирующая вероятностную природу встроенных этических ограничителей. В отличие от жёстких правил классического файрвола, фильтры LLM являются скорее высоковероятностной рекомендацией. Если атакующий находит такую комбинацию входных данных, которая смещает распределение вероятностей ответа в сторону запретной темы и смещение оказывается достаточным для преодоления порога срабатывания защитного классификатора, то ограничение рушится.

Не меньшую угрозу представляет проблема утечки информации (Data Leakage). Парадокс глубокого обучения заключается в том, что модель запоминает корреляции, но не факты. Однако при определённых условиях модель способна дословно воспроизвести фрагменты конфиденциальных текстов, на которых она обучалась (ключи API, персональная переписка или закрытые корпоративные документы). Смежная категория – это утечка системного промпта (System Prompt Leakage), при которой злоумышленник выманивает у модели её исходные инструкции, содержащие критически важную информацию о логике работы приложения.

Отдельный класс рисков связан с архитектурными особенностями современных систем, использующих поиск по векторным представлениям (Vector and Embedding Weaknesses). Модель опирается на внешние базы знаний, где документы хранятся в виде эмбеддингов. Уязвимости здесь бывают двух видов. Во-первых, злоумышленник способен отравить корпус документов таким образом, что при релевантном запросе модель будет подхватывать вредоносный контекст. Во-вторых, из-за ошибок в

разграничении доступа в общем векторном хранилище происходит утечка чужих данных через семантический поиск.

Отдельного рассмотрения заслуживает феномен галлюцинаций (Misinformation). На первый взгляд может показаться, что генерация моделью недостоверной информации — это проблема качества, а не безопасности. Однако уверенно изложенный вымысел в системах принятия решений способен нанести вполне реальный операционный или репутационный ущерб. С точки зрения злоумышленника, галлюцинации открывают вектор социальной инженерии. Атакующий может целенаправленно подталкивать модель к генерации ложных, но правдоподобных утверждений, которые затем используются для манипуляции пользователями сервиса. Особенно тревожно, что механизмы возникновения галлюцинаций до конца не изучены и коренятся в самой природе вероятностного предсказания следующего токена, что делает полное устранение этой уязвимости практически невозможным на текущем этапе развития технологии. Речь может идти лишь о снижении частоты и опасности таких инцидентов через RAG-архитектуры с верифицированными источниками и дополнительные слои фактчекинга.

Завершая обзор, необходимо упомянуть про избыточную агентность (Excessive Agency) и небезопасную обработку выходных данных (Improper Output Handling). Первая возникает, когда языковой модели предоставляется чрезмерная автономия и широкие полномочия на вызов внешних функций: отправку электронных писем, чтение файловой системы, выполнение кода или обращение к базам данных. Любая успешная промпт-инъекция превращается в действие с реальными последствиями. Формально злоумышленник манипулирует текстом, но по факту он получает возможность

удалённого выполнения команд на сервере. Вторая проявляется тогда, когда сгенерированный моделью текст без должной проверки передаётся на исполнение в другие компоненты системы. Если LLM генерирует фрагмент JavaScript-кода или SQL-запрос, и этот результат без должной санитизации передаётся на исполнение, последствия могут варьироваться от XSS-атаки в браузере клиента до полной компрометации базы данных.

Безопасность LLM не может быть обеспечена одним универсальным средством. Она должна проектироваться как сквозное свойство, пронизывающее архитектуру приложения на всех уровнях. По мере движения к системам непрерывного обучения сложность будет только возрастать, делая задачу построения безопасного искусственного интеллекта одной из центральных технологических проблем.

Методы защиты и повышения безопасности

Современные защитные механизме в области LLM уже невозможно описать какой-либо одной техникой или единственным алгоритмическим решением. На смену моностилистической парадигме безопасности пришла многоуровневая архитектура. Существующие подходы к защите целесообразно классифицировать на направленные на предотвращение атаки до момента генерации опасного контента и ориентированные на выявление аномалий или вредоносной активности в уже сгенерированном потоке.

Один из наиболее важных методов защиты ориентирован на данные модели, используемые для обучения. Здесь всё большее значение приобретают технические стандарты подтверждения подлинности, которые позволяют отслеживать и отсеивать потенциально скомпрометированные или синтетически сгенерированные фрагменты. Также ключевую роль играют

механизмы изоляции и скоупинга – модель должна иметь чёткие границы доверия к внешним источникам. Запросы к базам знаний должны выполняться в изолированной среде, а результаты перед внедрением в контекст должны проходить через фильтры, верифицирующие их релевантность и отсутствие скрытых инъекций команд.

Как уже отмечалось ранее, одним из подходов по обеспечению безопасности больших языковых моделей является выравнивание модели с помощью обучения с подкреплением. Однако статическое выравнивание страдает от эффекта, при котором модель, натренированная на фиксированном наборе сценариев, часто оказывается бессильной перед новыми, невиданными ранее формами запросов. В результате стало активно развивает направление динамической адаптации защиты без изменения весов. В частности, заметный интерес представляет метод Dynamic Deep Prompt Optimization. В отличие от статических префиксов безопасности, добавляемых к каждому запросу, данный подход использует промежуточные слои самой LLM в качестве экстракторов признаков для генерации уникального защитного эмбеддинга под каждый конкретный входной запрос. Этот эмбеддинг, сформированный легковесным перцептроном, внедряется в последующие слои модели, создавая контекстно-зависимый барьер, который эффективен против семантически неоднозначных вредоносных промптов.

Параллельно с этим развиваются методы защиты, при которых модель целенаправленно атакуют вредоносными запросами, а затем дообучают на успешно отражённых атаках, постепенно повышая её иммунитет.

Особого внимания заслуживает внешний контур защиты – премодерация входящих запросов и постмодерацию сгенерированных ответов. Однако злоумышленники демонстрируют способность обходить статические ограждения. Поэтому современные архитектуры безопасности всё чаще включают компоненты, основанные на агентном анализе. Агентно-ориентированные системы мониторинга анализируют всю траекторию поведения модели, что позволяет детектировать аномалии не по статичным сигнатурам, а по отклонениямв ходе выполнения задачи.

Следует признать, что каждое новое поколение защитных ограждений провоцирует разработку всё более изощрённых методов их обхода, и напротив – каждая громкая демонстрация уязвимости стимулирует появление новых контрмер. В этой связи важно подчеркнуть, что абсолютная безопасность LLM на сегодняшний день остаётся скорее недостижимым идеалом, нежели реальностью. Однако именно многоуровневый, эшелонированный подход, объединяющий гигиену обучающих данных, интернализацию безопасного поведения в весах модели и динамический контекстный надзор за её эксплуатацией, представляет собой наиболее зрелый и прагматичный ответ на существующий спектр угроз. Дальнейший прогресс в этой области будет связан с углублением интеграции методов защиты непосредственно в архитектуру самих моделей и вычислительных сред, в которых они функционируют.

Заключение

Завершая статью, следует подчеркнуть, что безопасность больших языковых моделей представляет собой непрерывный процесс, требующий постоянного внимания и итеративной адаптации. Многоуровневая архитектура защиты, объединяющая

контроль обучающих данных, выравнивание поведения модели и эксплуатационный мониторинг, сформировалась как прагматичный ответ существующий спектр угроз. Однако по мере того как LLM продолжают эволюционировать в сторону большей автономности и более глубокой интеграции с внешними системами, можно ожидать появления новых классов уязвимостей, которые потребуют переосмысления существующих защитных подходов. Дальнейшее развитие этой области, по всей видимости, будет двигаться в направлении более тесного сопряжения методов безопасности с самой архитектурой моделей, а также в сторону создания систем, способных не только реагировать на известные угрозы, но и упреждать возникновение новых. Именно эта траектория задаёт вектор будущих исследований и разработок в сфере безопасного применения генеративного искусственного интеллекта.

Список литературы

1. OWASP Top 10 for Large Language Model Applications // OWASP URL: https://owasp.org/www-project-top-10-for-large-language-model-applications/ (дата обращения: 18.04.2026).

2. Attack and defense techniques in large language models: A survey and new perspectives // ScienceDirect URL: https://www.sciencedirect.com/science/article/abs/pii/S0893608025012699?via%3Dihub (дата обращения: 18.04.2026).

3. A Comprehensive Survey in LLM (-Agent) Full Stack Safety: Data, Training and Deployment // ISI URL: https://www.isi.edu/results/publications/36415/a-comprehensive-survey-in-llm-agent-full-stack-safety-data-training-and-deployment/ (дата обращения: 18.04.2026).